



INDIAN INSTITUTE OF INFORMATION TECHNOLOGY SRI CITY

SOLUTION OF MIDSEM EXAMINATION – SPRING 2026

Data Structures and Algorithms

CSE/ECE:UG1/2/3/4(~~IE/IC/PE/PC~~)

Date: 26-02-2026

Duration: 90 Mins (02:30-4:00PM)

Max. Marks: 25

Instructions:

Roll No: _____

1. All questions are compulsory. **Name:** _____
2. Write the answers legibly. Cleanliness and legibility carries 1 mark.
3. Put a margin on each page towards the left. Within margin write clearly the question number. Begin each question's answer on a fresh page. Do not interleave one question's answer with that of the other question's answer. This can invalidate both answers.
4. Ordinary scientific calculators are allowed.
5. Attach the question paper to your answer book-let.

1	a. For the following c function, find out the time complexity analysis in terms of θ notation. <pre>#include <stdio.h> void harmonic_hopper(int n) { int count = 0; // Outer loop for (int i = 1; i <= n; i++) { // Inner loop // Note the starting point and the increment for (int j = i; j <= n; j += i) { count++; printf("Processing: %d, %d\n", i, j); } } }</pre>	[4 M]
---	--	-------

	Solution: The inner loop is executed, for each i, n/i times. So the total time, $T(n) = \frac{n}{1} + \frac{n}{2} + \frac{n}{3} + \dots + \frac{n}{n} = n \cdot \sum_{k=1}^n \frac{1}{k}$ [2 Marks] Note that, $\int_1^{n+1} \frac{1}{x} dx \leq \sum_{k=1}^n \frac{1}{k} \leq 1 + \int_1^n \frac{1}{x} dx$ i.e., $\ln(n+1) \leq \sum_{k=1}^n \frac{1}{k} \leq 1 + \ln n$ So, $\sum_{k=1}^n \frac{1}{k} \in \theta(\ln n)$ So, $n \cdot \sum_{k=1}^n \frac{1}{k} \in \theta(n \ln n)$ [2 Marks] So, the tight bound of the c function is $\theta(n \ln n)$.	
	b.Solve the given recurrence relation using the substitution method and express the time complexity using appropriate asymptotic notation. $T(n) = \begin{cases} 3T(n/2) + n, & n \geq 1 \\ 0, & n = 0 \end{cases}$	[4 M]

(b) $T(n) = \begin{cases} 3T(\frac{n}{2}) + n & , n \geq 1 \\ 0 & , n = 0 \end{cases}$

Assume $n = 2^k$, Expand the relation using substitution.

→ expand $T(n) = 3 \left[3T(\frac{n}{4}) + \frac{n}{2} \right] + n$

$$= 3^2 T(\frac{n}{4}) + 3 \cdot \frac{n}{2} + n$$

Substitute again

$$= 3^2 \left[3T(\frac{n}{8}) + \frac{n}{4} \right] + 3 \cdot \frac{n}{2} + n$$

$$= 3^3 T(\frac{n}{8}) + 3^2 \cdot \frac{n}{4} + 3 \cdot \frac{n}{2} + n$$

⋮

After k expansions,

$$T(n) = 3^k T(\frac{n}{2^k}) + n \left[1 + \frac{3}{2} + \left(\frac{3}{2}\right)^2 + \dots + \left(\frac{3}{2}\right)^{k-1} \right]$$

G.P

Since $n = 2^k \Rightarrow k = \log_2 n$

$\therefore \log_a n = n^{\log_a b}$

$$3^{\log_2 n} \cdot T(\frac{2^k}{2^k})$$

$$= n^{\log_2 3} \cdot T(1)$$

$$n \left[\frac{\left(\frac{3}{2}\right)^k - 1}{\left(\frac{3}{2} - 1\right)} \right]$$

$$n \left[\frac{\left(\frac{3}{2}\right)^{\log_2 n} - 1}{(1/2)} \right]$$

$$\approx n \cdot \left(\frac{3}{2}\right)^{\log_2 n}$$

~~$T(n) = n^{\log_2 3} \cdot T(1)$~~

$$= n \cdot n^{\log_2 \frac{3}{2}}$$

$$= n^{1 + \log_2 \frac{3}{2}}$$

$$\approx n^{\log_2 2 + \log_2 \frac{3}{2}}$$

$$= n^{\log_2 3}$$

$$\therefore T(n) \approx n^{\log_2 3} T(1) + n^{\log_2 3}$$

$$\therefore T(n) = \Theta(n^{\log_2 3})$$

$\therefore$ Asymptotic complexity Θ

$$T(n) = \Theta(n^{\log_2 3})$$

4 Marks ; partial marking can be given according to procedure, correct steps, & final answer.

2	a. Let $f(n) = n^{0.001}$ and $g(n) = \log_e n$. Compare $f(n)$ and $g(n)$. That is, which of the following are true? (i) $f(n)$ is in $O(g(n))$, (ii) $f(n)$ is in $o(g(n))$, (iii) $f(n)$ is in $\Omega(g(n))$, (iv) $f(n)$ is in $\omega(g(n))$, and (v) $f(n)$ is in $\Theta(g(n))$. For each of your answers, you need to give a valid (mathematically sound) explanation(s) which are precise. Please do not write unnecessarily anything. Unnecessary explanations can attract negative marks. Solution: Consider $f(n)/g(n)$ as $n \rightarrow \infty$. Since this is in the form ∞/∞, we can employ the L'Hopitals rule. Differentiating numerator and denominator we get $f'(n)/g'(n) = 0.001n^{0.999} (n) = \infty$ as $n \rightarrow \infty$. So, $f(n)$ is in $\Omega(g(n))$ and also $f(n)$ is in $\omega(g(n))$. That is (i) is false (ii) is false (iii) is True [2 Marks] (iv) is True [2 Marks] (v) is false	[4 M]
	b. A library maintains book IDs in ascending order using a singly linked list. Initially, the IDs are: 101, 125, 210, 315, 420. Write a C program to insert a new book ID: 250 into the correct position such that the list remains sorted. Display the list before and after insertion. <pre>#include <stdio.h> #include <stdlib.h> struct Node { int data; struct Node* next; }; int main() { // Assume list already exists: 101 -> 125 -> 210 -> 315 -> 420, and // head points to it struct Node* head; // given starting address struct Node* temp; // Display before insertion printf("Before insertion:\n"); temp = head; while (temp != NULL) { printf("%d -> ", temp->data); temp = temp->next; } // Insert new element 250 struct Node* newNode = (struct Node*)malloc(sizeof(struct Node)); newNode->data = 250; newNode->next = NULL; temp = head; while (temp->next != NULL && temp->next->data < 250) { temp = temp->next; } }</pre>	[4 M]

```

newNode->next = temp->next;
temp->next = newNode;
// Display after insertion
printf("After insertion:\n");
temp = head;
while (temp != NULL) {
    printf("%d -> ", temp->data);
    temp = temp->next;
}
}
    
```

Partial marks can be given according to extent of correctness.

Note: If they assume the list with elements already exists or if they create the list with the given elements, and insert the new element 250, both can be given marks.

3

a. Find Postfix expression of the Infix expression:
 $A+B*(C+D-E)/F-(D+A)*E^{(B+G)}$
Show your computation in the form of a table.

Solution: The given infix expression: $A+B*(C+D-E)/F-(D+A)*E^{(B+G)}$.
Computation is shown in the following table.

S.No	Current Symbol	Operator Stack	Postfix Expression
1	A		A
2	+	+	A
3	B	+	AB
4	*	+	AB
5	(	+(	AB
6	C	+(	ABC
7	+	+(+	ABC
8	D	+(+	ABCD
9	-	+(	ABCD+
10	E	+(	ABCD+E
11	)	+	ABCD+E-
12	/	+/	ABCD+E-*
13	F	+/	ABCD+E-*
14	-	-	ABCD+E-*/+

[4 M]

15	(	-(	ABCD+E-*F/+
16	D	-(	ABCD+E-*F/+D
17	+	-(+	ABCD+E-*F/+D
18	A	-(+	ABCD+E-*F/+DA
19	)	-	ABCD+E-*F/+DA+
20	*	_*	ABCD+E-*F/+DA+
21	E	_*	ABCD+E-*F/+DA+E
22	^	_*^	ABCD+E-*F/+DA+E
23	(	_*^(	ABCD+E-*F/+DA+E
24	B	_*^(	ABCD+E-*F/+DA+EB
25	+	_*^(+	ABCD+E-*F/+DA+EB
26	G	_*^(+	ABCD+E-*F/+DA+EBG
27	)	_*^	ABCD+E-*F/+DA+EBG+
28			ABCD+E-*F/+DA+EBG+^*_-
So, the postfix expression is ABCD+E-*F/+DA+EBG+^*_- [4 Marks] Partial marks can be given according to the number of correct steps.			
b. Trace the execution of the following stack program. Show the value of <i>top</i> and the stack contents after each operation. Illustrate the stack states clearly at each step and determine the final output. <pre> #include <stdio.h> int top = -1; int stack[6]; void push(int x) { stack[++top] = x; } int pop() { return stack[top--];} int main() { push(5); push(10); push(15); stack[top-1] = pop(); push(stack[top] * 2); printf("%d %d %d", stack[0], stack[1], stack[2]); return 0; }</pre>			[4 M]

Code	Top	Stack
int top = -1; int stack[6];	top=-1	stack = [_, _, _, _, _, _]
push(5) => stack[++top] = 5;	top=0	stack = [5, _, _, _, _, _]
push(10) => stack[++top] = 10;	top=1	stack = [5, 10, _, _, _, _]
push(15) => stack[++top] = 15;	top=2	stack = [5, 10, 15, _, _, _]
stack[top-1]=pop() Evaluate from Right to Left: 1. pop() => return 15 2. stack[top-1] = 15 => stack[0] = 15 (OR) Evaluate from Left to Right: 1. Evaluate stack[top-1] 2. pop() => return 15 3. stack[top-1] = 15 => stack[1] = 15	 top = 1 top = 1 top=2 top=1 top=1	 stack = [5, 10, _, _, _, _] stack = [15, 10, _, _, _, _] target stack[1] stack = [5, 10, _, _, _, _] stack = [5, 15, _, _, _, _]
push(stack[top]*2) => stack[++top] = 10*2; (OR) stack[++top] = 15*2;	top=2	stack = [15, 10, 20, _, _, _] (OR) stack = [5, 15, 30, _, _, _]
printf("%d%d%d", stack[0], stack[1], stack[2]);		Prints: 15, 10, 20 (OR) Prints: 5, 15, 30
So, the final output of the program is: 15, 10, 20 (OR) 5, 15, 30 depending on how the compiler evaluate i.e., Right to Left or Left to Right [4Marks] Note: Partial marks can be given according to the number of correct steps.		